

zilla-kb-tool — Complete Guide

Automated knowledge-base regression testing for Zilla voice agents • v2026-08-22

What it does. End-to-end pipeline that turns a Word-document knowledge base into realistic test questions (Arabic & English), speaks them to live Zilla voice agents through real calls or WebSockets, records the answers, and grades every answer with an LLM judge on **Relevancy** and **Faithfulness** - producing auditable score reports.

TypeScript (tsx)

Python evaluator

DeepEval 4.1.8

Playwright call runner

WebSocket runner

Edge TTS + ffmpeg

Gemini question generator

Groq/OpenRouter/Ollama judges

1. Pipeline at a glance

```
spl_kb.docx
| npm run extract-chunks      (python-docx)
v
data/chunks.json --> npm run generate-questions (Gemini) --> data/questions.json
|
| npm run validate-questions
v
console report

                                npm run generate-audio
                                v (Edge TTS + ffmpeg)
                                wav/Qxxx.wav (+ audio_file)
                                |
                                npm run build-sessions <-----+
                                | (seeded shuffle, AR+EN sessions)
                                v
                                data/sessions.json
                                |
                                npm run run-sessions (browser call via Playwright "CallRunner")
                                OR run-sessions-ws (direct WebSocket, text turns)
                                v
                                data/runs/<session>/... + data/runs/run-results-<stamp>.json
                                | npm run capture-answers
                                v
                                data/captured-answers-<stamp>.json
                                | npm run evaluate (DeepEval judge)
                                v
                                data/eval-results-*.json -> reports

sidecars: npm run agents (resolve agent names -> ids in .env)
          npm run clean  (--from N resets phases N..8)
          npm run ui      (local dashboard for everything above)
```

2. Phase reference

Phase 1 **extract-chunks** — `src/question-generator/extract_chunks.py`

Parses the KB Word document into addressable chunks: one chunk per non-empty paragraph, one per table row (merged-cell duplicates removed). Tags each chunk with stable id (`chunk_001...`), language (`ar / en` by Unicode range) and group.

Input	<code>spl_kb.docx</code>
Output	<code>data/chunks.json</code> - <code>{id, type, text, language, group}</code>
Services	none (local python-docx)

Phase 2 **generate-questions** — `src/question-generator/generate.ts`

Picks chunks (standalone + pairs from the same table for "combined" questions) and asks an LLM for one realistic customer question + ground-truth answer each, in EN and AR (same `pair_id`). Resumable, rate-limit-aware, saves after every question. Short chunks (<15 chars) skipped.

Input / Output	<code>data/chunks.json</code> → <code>data/questions.json</code> (<code>{id Q###, pair_id P###, language, question, expected_answer, source_chunks[]}</code>)
Services	Gemini <code>gemini-3.6-flash</code> or OpenAI-compatible (temperature 0.3, JSON mode)
Key env	<code>SAMPLE_SIZE</code> (number all), <code>GENERATE_ENGLISH/ARABIC</code> , <code>RESUME</code> , <code>MAX_RETRIES</code> , <code>REQUEST_DELAY_MS</code>

validate-questions re-checks any questions file (ids, uniqueness, languages, non-empty fields, source chunks resolvable against `chunks.json`). Console report only.

Phase 3 **generate-audio** — `src/tts/generate_audio.ts`

Speaks every question: Edge TTS (voices `ar-EG-SalmaNeural` / `en-US-JennyNeural`) → mp3 → ffmpeg → 16 kHz mono WAV; writes `audio_file` back into `questions.json`. Resumable, 3 retries.

Phase 4 **build-sessions** — `src/session-builder/build_sessions.ts`

Bundles questions that HAVE audio into multi-turn sessions per language using a seeded deterministic shuffle (mulberry32, seed `RANDOM_SEED=42`) so runs are reproducible.

Output	<code>data/sessions.json</code> - <code>{session_id S###_ar/_en, language, agent_id, messages[{turn, question_id, audio_file}...]}</code>
Key env	<code>MESSAGES_PER_SESSION=10</code> , <code>TOTAL_SESSIONS_AR/EN=10</code> , <code>ALLOW_REPEAT_QUESTIONS</code> , <code>ZILLA_AGENT_ID_AR/EN</code>

Phase 5 **running sessions** — two interchangeable runners

	run-sessions.ts (UI mode)	run-sessions-ws.ts (direct WS)
Mechanism	Playwright browser drives the real Zilla web app, injects WAV clips as mic input (vendored CallRunner subprocess)	One raw WebSocket to <code>WS_URL</code> <code>.../audio/{token}</code> ; sends <code>{"text": question}</code> , assembles streamed reply sentences
Needs	Zilla email/password login, staged clips	No browser, no audio
Turn-complete logic	CallRunner e2e heuristics	1800 ms frame silence (settle) + 20 s hard timeout
Outputs	Identical shapes: per-session <code>summary-<stamp>.json</code> + aggregate <code>data/runs/run-results-<stamp>.json</code>	
Scheduling env	<code>CONCURRENCY</code> , <code>CONCURRENT_AR/EN</code> , <code>--sessions S001_ar,...</code> ; UI mode adds <code>RESHUFFLE/SHUFFLE_SESSIONS/SESSIONS_PER_RUN</code>	

Phase 6 capture-answers — `src/capture/capture_answers.ts`

Joins raw run artifacts into the clean dataset the evaluator consumes: customer question → following agent answer, plus expected answer, source chunks, latency and call URL. Question order is taken from staged clip filenames (authoritative under reshuffle).

Input newest `run-results-*.json` (+ summaries, sessions, questions)

Output `data/captured-answers-<ISO-stamp>.json` with totals (sessions, answered/unanswered, avg latency) and per-turn records

Phase 7 evaluate — `src/evaluator/evaluate.py`

DeepEval-based grading of every captured answer:

- **Adapter** `OpenAICompatModel` : plain chat-completions client so ANY OpenAI-compatible endpoint works - Groq cloud, OpenRouter cloud, or local Ollama. Temperature 0, key chosen by endpoint host. Counts tokens/calls per run.
- **Dialect-aware templates**: wrappers adding our rules to DeepEval prompts: Levantine-dialect tolerance, Arabic-native comparison (no translation), English-only reasons, no-truncation claim extraction, "فقط" list-scope rule, natural-closings rule.
- **Metrics**: Answer-Relevancy + Faithfulness (threshold 0.5); deterministic reasons rebuilt from raw verdict counts instead of LLM prose.
- **Resilience**: auto-retry (3×) on HTTP 429, malformed JSON and empty completions; results saved after EVERY question so resume never loses work; result files are named per source AND per judge so switching judges never clobbers scores.
- **Observability**: per-question judge latency + run-level token usage stored in output.

3. Sidecar tools

Command	What it does
---------	--------------

<code>npm run agents</code>	Headless-browser login to Zilla, lists account agents, resolves agent NAMES to IDs (case-insensitive unique substring match) and rewrites <code>ZILLA_AGENT_ID_AR/EN</code> in <code>.env</code> . <code>--list</code> / <code>--json</code> flags.
<code>npm run clean - --from N</code>	Resets generated data from phase N onward (1=chunks ... 8=eval results); <code>--dry</code> lists without deleting. Never touches <code>spl_kb.docx</code> .
<code>npm run ui</code>	Local dashboard on <code>127.0.0.1:5173</code> : edit every <code>.env</code> value, run/stop any phase with live console, status cards, animated per-call transcripts with metric pills. No external deps (node:http only).

4. Data dictionary

File	Produced by	Consumed by
<code>spl_kb.docx</code>	(source KB)	extract-chunks
<code>data/chunks.json</code>	extract-chunks	generate-questions, validate
<code>data/questions.json</code>	generate-questions (+TTS update)	audio, sessions, capture, UI
<code>wav/Qxxx.wav</code>	generate-audio	build-sessions filter, UI runner
<code>data/sessions.json</code>	build-sessions	both runners, capture, UI
<code>data/runs/**</code>	runners	capture-answers, UI
<code>data/captured-answers-*.json</code>	capture-answers	evaluate, UI
<code>data/eval-results-*.json</code>	evaluate	UI, reports

5. Configuration (.env groups)

[question generation]	LLM_PROVIDER=gemini, GEMINI_MODEL=gemini-3.6-flash, GEMINI_API_KEY, SAMPLE_SIZE, GENERATE_ENGLISH/ARABIC, RESUME, MAX_RETRIES
[judge]	EVAL_LLM_PROVIDER=openai, EVAL_MODEL=openai/gpt-oss-120b, EVAL_LLM_BASE_URL=https://api.groq.com/openai/v1, GROQ_API_KEY (alt: OpenRouter sk-or-v1-... key in OPENAI_API_KEY, or Ollama local)
[sessions]	MESSAGES_PER_SESSION=10, TOTAL_SESSIONS_AR/EN=10, RANDOM_SEED=42, ALLOW_REPEAT_QUESTIONS, ZILLA_AGENT_ID_AR/EN
[zilla connection]	APP_URL=https://ziila-stg.intella.me, VITE_API_BASE_URL=...api, ZILLA_EMAIL/ZILLA_PASSWORD
[runner scheduling]	CONCURRENCY=2, CONCURRENT_AR/EN, SESSIONS_PER_RUN, SHUFFLE flags
[websocket runner]	WS_URL=ws://host:port/audio/{token}, WS_MODE=text, WS_SETTLE_MS=1800, WS_TURN_TIMEOUT_MS=20000, WS_INTER_TURN_MS=400
[dashboard]	UI_PORT=5173

6. Quality assurance of the judge itself

The judge was selected by grading the SAME 5 Arabic questions with three candidate models, then stress-tested with deliberately corrupted answers (poison test):

Judge	Relevancy	Faithfulness	Time/5q	Verdict
gpt-oss-120b (Groq)	1.00	1.00	2.7 min	adopted
nemotron-550b (OpenRouter)	0.72	0.90	10.1 min	strong but slow
qwen2.5:7b (local)	0.62	0.57	~3 min	offline fallback only

Poison test: 4/4 correct. Fake cities → Faithfulness 0.00; fabricated numbers (999) → Faithfulness 0.00; off-topic chatter → Relevancy 0.00; untouched control answer → perfect scores. Each metric catches exactly its own fault type.

7. Quickstart

```
npm install && pip install deepeval python-dotenv openai docx edge-tts
npm run agents -- --list          # pick agent ids -> .env
npm run extract-chunks && npm run generate-questions && npm run validate-questions
npm run generate-audio && npm run build-sessions
npm run run-sessions-ws          # or run-sessions for real browser calls
npm run capture-answers && npm run evaluate -- --sample 5 --no-resume
# or drive everything from: npm run ui
```

Quota cheat-sheet. Groq (judge): ~1,000 requests per rolling ~2h45m window - a 200-question run (~1,400 calls) needs two sittings or let retry+resume bridge the reset. OpenRouter free: ~50 requests/day total. Gemini (questions): standard free limits. Local Ollama: unlimited, lower accuracy on long-list comparisons.